BIG DATA & CLOUD COMPUTING / FINAL PROJECT

em lyon business school

Predicting Arrests in Chicago

What 239,548 incident reports can tell us - and what they cannot.

11.7%
of reported incidents lead to an arrest

Research question
Which incident characteristics are associated with a recorded arrest, and how accurately can Spark MLlib predict one?

Chicago Police Department
2022 · Kaggle

Alicia Öhrn Ibarra Ethan Le Chevallier Martin Rabier

Chicago is more than a nickname

Real challenges deserve evidence - not a single story about the city

2.7M

residents

2022 U.S. Census estimate

239,548

incident reports

Chicago · 2022 project dataset

CULTURE AND CONTEXT

“Chiraq” Chicago + Iraq

The portmanteau emerged from local hip-hop and drill, then entered wider pop culture through music and Spike Lee’s 2015 film.

Its meaning is contested: it can voice lived experience and neglect, but can also reduce a complex city to a war-zone image.

Why this analysis? Examine where recorded arrest outcomes differ - without treating arrest rates as measures of safety, solvability, or a prescription for enforcement.

Sources: U.S. Census Bureau (2022 estimate; revised 2024); The Guardian (2015); Chicago Magazine (2015).

Data and pipeline

239,558 reports in - one end-to-end Spark pipeline

239,558
incidents
22 columns · Chicago · 2022

239,548
rows retained
10 rows with missing Ward removed

Why Spark?
Aggregation, cleaning, and training run in one distributed pipeline that can scale to multiple years of records.



Targeted cleaning

Dropped Year, Updated On, and Case Number (239,511 distinct values). Removed coordinates (missing in 4,667 rows) and street numbers; retained roughly 2,460 street names.

Missing values

Removed 10 rows missing Ward; filled 965 missing locations with the mode (STREET). Runtime: Databricks.

Two lenses, one question

From descriptive insights to prediction

01 Describe

Which incident types, locations, and times are associated with recorded arrests?

→

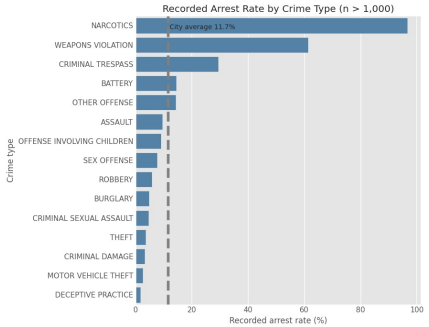
02 Predict

Can the available features outperform a simple, transparent rule?

First, what do the data tell us?

Crime type is the dominant signal

Recorded arrest rates vary sharply by offense



Recorded arrest rate by crime type.

CITYWIDE AVERAGE

11.7%

96.7% Narcotics

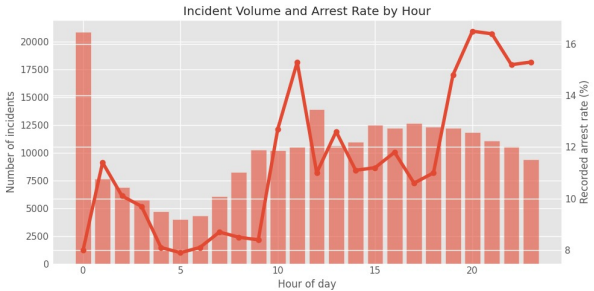
61.4% Weapons

29.5% Criminal trespass

The workload gap: four property crimes account for **50.3% of incidents** (120,447), but only **3.2% lead to arrest** (3,802).

Time and place add weaker signals

Timestamps and raw comparisons need caution



Incident volume (bars) and recorded arrest rate (line) by hour.

Weekday: nearly flat, 11.0% Friday to 12.3% Sunday. **Domestic:** 12.6% vs. 11.5% (+1.1 percentage points).

7.9–8.1%

Early morning · 4–6 a.m.

recorded arrest rate

14.8–16.5%

Evening · 7–11 p.m.

recorded arrest rate

Public: sidewalk 23.4%
Residence: 6.7%

Midnight is a suspicious spike: 20,874 reports at 12 a.m. vs. 7,630 at 1 a.m. Default timestamps may distort the pattern.

Districts and proactive offenses

Two cautions before reading raw rates

RAW ARREST RATE

7.6% - 23.2%

A 3× difference across districts

AFTER CRIME-MIX ADJUSTMENT

0.80 - 1.40

Observed / expected arrests, by crime type.

Takeaway: compare districts after adjustment, never on raw rates alone.

7.4%

of incidents

40.1%

of arrests

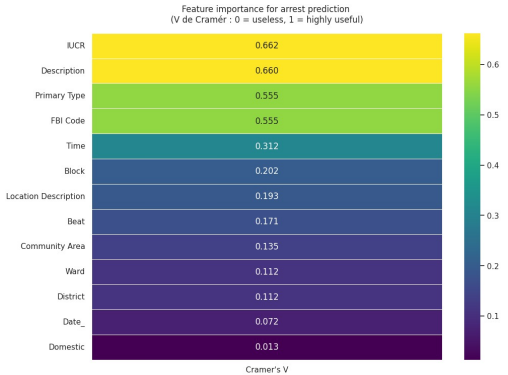
Narcotics, weapons, and criminal trespass.

A partly circular result

These proactive offenses are often detected through the arrest itself; predicting arrest can therefore be partly circular.

Preparing the prediction task

Rare target, categorical features, held-out evaluation



Cramér's V candidate screen; redundant FBI Code and IUCR excluded.

88 / 12

Class imbalance

88.3% without arrest · 11.7% with arrest

Handling

Used inverse-frequency class weights. SMOTE is not available in MLlib, and interpolating encoded categories would not be meaningful.

Protocol

Random 70/30 split, seed 42. Test set: **72,090 incidents**, including **8,497 arrests**. Default Spark MLlib models.

Model results

GBT leads MLlib - but accuracy tells a different story

Model	Precision	Recall	F1	AUC-ROC	AUC-PR
Decision Tree	0.300	0.687	0.417	0.714	0.499
Random Forest	0.337	0.641	0.442	0.831	0.567
GBT	**0.426**	**0.653**	**0.515**	**0.844**	**0.635**

Arrest-class precision $\frac{TP}{TP + FP}$

Among incidents flagged for arrest, how many have a recorded arrest? TP = true positives; FP = false positives.

Arrest-class recall $\frac{TP}{TP + FN}$

Among incidents with a recorded arrest, how many did the model find? FN = false negatives.

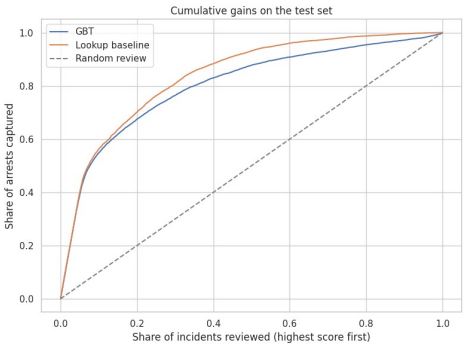
0.43 precision
0.65 arrest recall

In practice: about 1.4 false alarms per arrest found.
GBT accuracy is 85.5%, below the 88.2% "never arrest" rule - a misleading metric for an imbalanced target.

We focus on arrest-class precision/recall, AUC-ROC, and AUC-PR - not accuracy alone.

The twist: the baseline wins

A simple historical lookup beats all three MLlib models



Cumulative share of arrests captured by reviewing the highest-scored incidents first.

AUC-ROC

0.885

Lookup by crime type × description

VS. **0.844** GBT

70% vs. 67%

Arrests captured

Top 20% of incidents reviewed; random: 20%.

Robustness · victim-reported incidents (93%)

Baseline AUC-ROC: **0.823** · GBT: **0.763**.

Business implications and responsible use

Useful indicators - not an automated decision

Recommendations

- Track **incident volume and arrest rate** together, by category.
- Compare districts using rates **adjusted for crime mix**.
- Prefer a **transparent, auditable rule** unless richer data demonstrates a clear gain.
- Examine high-volume, low-arrest contexts, such as early mornings and residential settings.

Limitations to keep in mind

One city and one year; random rather than temporal split; untuned models; potentially unreliable timestamps. Indexers and Cramér's V were computed before the split, creating a leakage risk to address in future validation. Geography may encode reporting and enforcement patterns; associations are not causal.

Responsible use

A score does not prove that an arrest will happen - or that it should happen.

A score may inform human judgment; it must never replace it.

CONCLUSION

Here, the simplest rule is the most useful.

Spark enabled a scalable end-to-end analysis. But added model complexity did not improve predictive value.

0.885
Baseline AUC-ROC
type × description lookup

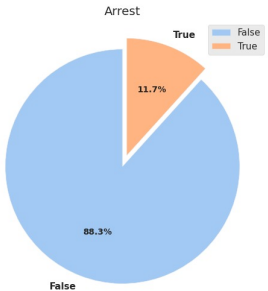
0.844
GBT AUC-ROC
best MLlib model

100%
Human judgment
always central

Thank you.

Appendix A · The target is highly imbalanced

Why accuracy alone can be misleading



88.3%

No recorded arrest

211,498 of 239,548 incidents

11.7%

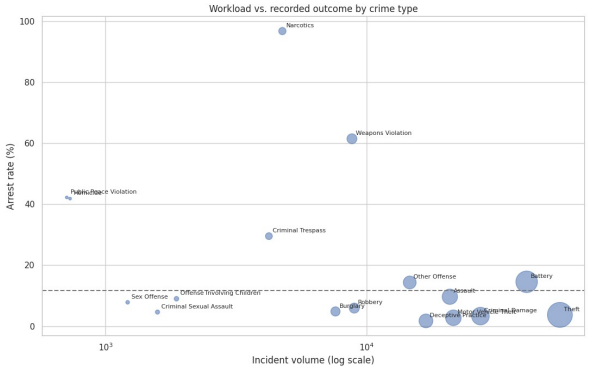
Recorded arrest

28,050 of 239,548 incidents

A classifier that always predicts “no arrest” reaches about 88.3% accuracy while detecting **zero arrests**. This is why the analysis uses class weights and arrest-class metrics.

Appendix B · Incident volume versus arrest rate

Read workload and outcome together



Each point represents a crime type; incident volume is shown on a logarithmic scale.

120,447

incidents across four property-crime categories

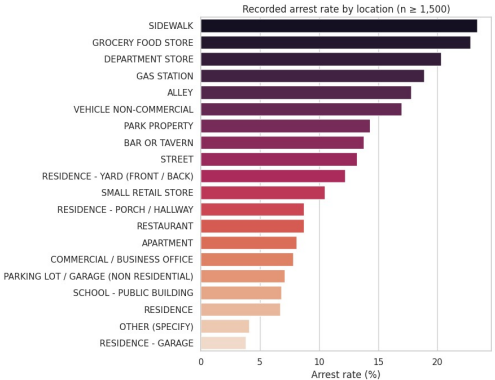
3.2%

recorded arrest rate

Rare, high-rate offenses and common, low-rate offenses have very different operational footprints.

Appendix C · Recorded arrest rates by location

Public settings are higher; residential settings are lower



23.4%

Sidewalk

12,059 incidents

22.8%

Grocery store

Public/commercial location

6.7%

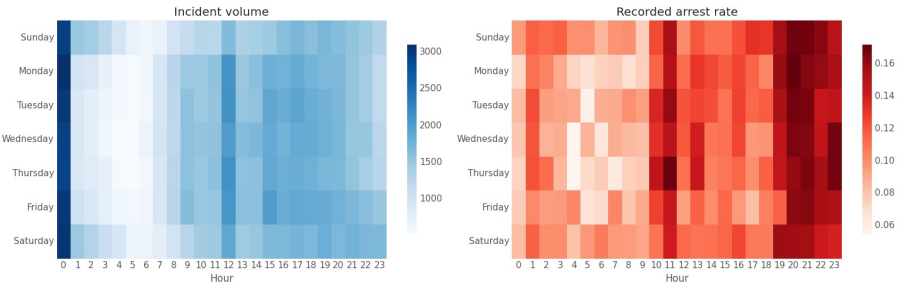
Residence

8.1% for apartments

Domestic: 12.6% vs. 11.5% (+1.1 pp) — statistically significant, but modest.

Appendix D · Weekday and monthly patterns

Small calendar differences; month-to-month rates vary more

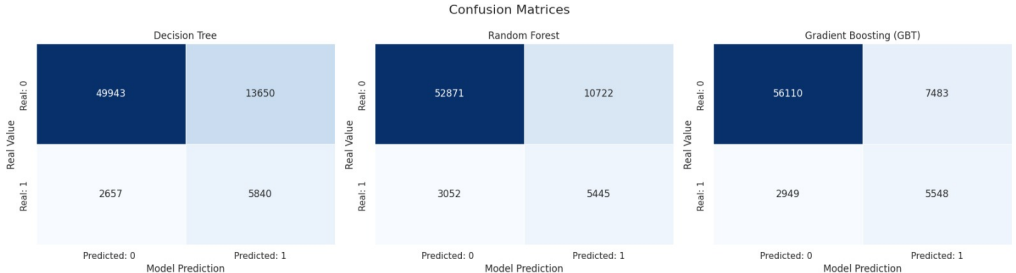


Weekday: 11.0% on Friday to 12.3% on Sunday - a weak signal.

Month: highest in May (14.3%); lowest in October–November (9.7–9.8%).

Appendix E · Model confusion matrices

GBT finds fewer false alarms than the other MLlib models



Model	False alarms	Precision	Recall
Decision Tree	13,650	0.300	0.687
Random Forest	10,722	0.337	0.641
GBT	**7,483**	**0.426**	**0.653**

All models use the same 72,090-incident test set (8,497 recorded arrests). Metrics refer to the arrest class.

Appendix F · Features and validation notes

What entered the models - and what a stronger evaluation should change

Candidate features retained

Primary Type, Description, Location Description, Block, Beat, Ward, District, Date, Time, and Domestic.

Screening: Spark ChiSquareTest, converted to Cramér's V. ID, IUCR, FBI Code, and Community Area were removed as identifiers or redundant fields.

Training and evaluation

Inverse-frequency class weights; random 70/30 split (seed 42); default Decision Tree, Random Forest, and GBT settings; no cross-validation or hyperparameter tuning.

Validation caveat: feature screening and category indexing were performed before the split. Refit these steps inside each training fold, then use a temporal holdout and multi-year data to test generalization.

Appendix G · Turning columns into model inputs

What the preparation code does, step by step



Notebook cell 75: indexing, vector assembly, class weights, and split.

Leakage to fix

Indexers and the earlier Cramér's V screen use all rows. Split first, then fit screening and preprocessing on training data only.

1 · Encode categories

`StringIndexer` maps text labels to category indices. `handleInvalid="keep"` reserves an index for unseen values.

2 · Assemble features

`VectorAssembler` packs indexed columns into one `features` vector for MLlib.

3 · Balance class influence

Weight $= N/(2N_{class})$. Each class therefore contributes about half the total training weight.

4 · Hold out test data

`randomSplit([0.7,0.3], seed=42)` creates a reproducible 70/30 split.

Appendix H · Training and checking the models

How the notebook turns training data into comparable results



```
dt = DecisionTreeClassifier(featuresCol="features", labelCol="label", weightCol="weight", seed=42,maxBins=3000)
rf = RandomForestClassifier(featuresCol="features", labelCol="label",weightCol="weight", seed=42,maxBins=3000)
gbt = GBTClassifier(featuresCol="features", labelCol="label", weightCol="weight", seed=42,maxBins=3000) #spark native alternative for XGBoost

models = [
    ("Decision Tree", dt),
    ("Random Forest", rf),
    ("Gradient Boosting (GBT)", gbt)
]

#Performance metrics
eval_f1 = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="f1")
eval_acc = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="accuracy")
eval_prec = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="weightedPrecision")
eval_rec = MulticlassClassificationEvaluator(labelCol="label", predictionCol="prediction", metricName="weightedRecall")
eval_auc = BinaryClassificationEvaluator(labelCol="label", rawPredictionCol="rawPrediction", metricName="areaUnderROC")

results = []
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle("Confusion Matrices", fontsize=16)
print("Training the models")

for i, (name, algorithm) in enumerate(models):
    print(f"Undergoing training - {name}...")
    model = algorithm.fit(train_data)
    predictions = model.transform(test_data)
    results.append({
        "Model": name,
        "Accuracy": round(eval_acc.evaluate(predictions), 4),
        "F1-Score": round(eval_f1.evaluate(predictions), 4),
        "Precision": round(eval_prec.evaluate(predictions), 4),
        "Recall": round(eval_rec.evaluate(predictions), 4),
        "AUC-ROC": round(eval_auc.evaluate(predictions), 4)
    })
    cm_spark = predictions.groupBy("label", "prediction").count().toPandas()
    cm = pd.DataFrame([[0, 0], [0, 0]], index=["Real: 0 ", "Real: 1 "], columns=["Predicted: 0 ", "Predicted: 1"])
    for _, row in cm_spark.iterrows():
        r = int(row["label"])
        c = int(row["prediction"])
        cm.iloc[r, c] = int(row["count"])
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False, linecolor='0.5')
    axes[i].set_title(name)
    axes[i].set_ylabel("Real Value")
    axes[i].set_xlabel("Model Prediction")
results_df = pd.DataFrame(results)
display(results_df)
plt.tight_layout()
```

Notebook cell 76: model setup, fit/transform loop, metrics, and confusion matrices.

Metric note

This cell averages precision and recall across both labels. Reported arrest-class metrics select label 1: precision 0.426, recall 0.653, F1 0.515.

1 · Define three learners

Decision Tree, Random Forest, and Gradient-Boosted Trees use the same features, label, and class weights. Seed 42 makes random choices reproducible; `maxBins=3000` allows many category levels.

2 · Fit, then predict

`fit(train_data)` learns from training rows. `transform(test_data)` scores held-out rows.

3 · Read the confusion matrix

Rows are actual labels; columns are predictions. The four cells count true negatives (no arrest correctly predicted), false positives, false negatives, and true positives.

Appendix I · How the crime-mix adjustment works

Compare each district with the arrest rate expected from its offense mix

STEP 1 · CITYWIDE RATE BY CRIME TYPE

$$p_t = \frac{\text{arrests of type } t}{\text{incidents of type } t}$$

Example: use the 2022 citywide arrest rate for each Primary Type.

STEP 2 · EXPECTED ARRESTS IN DISTRICT

$$E_d = \sum_{i \in d} p_{t_i}$$

Give each incident its crime-type rate, then add them for district d.

STEP 3 · OBSERVED / EXPECTED

$$Index_d = \frac{O_d}{E_d}$$

O_d is the actual arrest count. Values above 1 are above the crime-mix expectation; below 1 are below it.

Worked example: 1.20

If a district has **12 observed arrests** and its incident mix implies **10 expected**, then $12/10 = 1.20$: 20% above expectation, given crime type alone. This is an index, not an arrest probability.

How the notebook computes it

Calculate `avg(Arrest)` by `Primary Type`; join that rate back to every incident; group by `District`; sum actual arrests and summed type-specific rates; divide observed by expected. Districts with fewer than 1,000 incidents are excluded.

What the results mean The adjusted indices range from 0.80 (District 3) to 1.40 (District 1); District 18 is 1.27. This controls only for Primary Type; it is descriptive, not a causal measure of district performance.